

Lab Worksheet: Session 4

Class Relationships: Aggregation, Composition, and Dependency

Course	Object-Oriented Programming Lab (RTI253008)
Session	4 (Week 4)
Duration	1 × 4 × 50' lab work; 1 × 1 × 50' independent assignment/report
Sub-CPMK (Learning Outcome)	Implement relationships between classes (has-a): aggregation, composition, dependency (uses-a), and multiplicity, in Java code
Starting Code	no starting code required; each experiment begins from a new, self-contained package (you may continue the project from the previous worksheet)
Final Code	a project containing 6 packages, classrelation.experiment1 through experiment6, each with an implementation of Aggregation, Composition, and Dependency

A. Learning Outcomes

After finishing this worksheet, students will be able to:

1. Explain the difference between Association, Aggregation, Composition, and Dependency (uses-a)
2. Implement an Aggregation relationship between classes for a one-to-one case, a multiple relationship (one class related to more than one other class), and a relationship with several roles pointing to the same class.
3. Implement a Composition relationship, including the one-to-many pattern using an array of objects, and understand why “no setter” is an important part of the design.
4. Implement a Dependency (uses-a) relationship through a method parameter, and tell it apart from Aggregation.
5. Translate a UML class diagram – including the diamond notation ($\diamond/\blacklozenge$), dashed lines, and multiplicity – into Java code that is correct and consistent with the diagram.

B. Preparation and Prerequisites

- **Tools:** JDK 17 or newer, VSCode (the editor we use throughout this lab).
- **Prerequisites:** you have finished the classes-and-objects worksheet (defining classes, attributes, constructors, and methods, and understanding that an object variable is a reference, not the object itself — this idea is used a lot in this worksheet).
- **Project:** each experiment uses a new package named `<identifier>.classrelation.experimentN` (replace `<identifier>` with your own identity, for example `ac.id.polinema`). A separate package per experiment keeps classes with the same name (e.g. Car in Experiment 2 and Experiment 5) from clashing.

 **Info:** Package naming convention: the institution's domain name is written in reverse, so “polinema.ac.id” becomes `ac.id.polinema`, and then `.classrelation.experimentN` is added.

Here is a map of the relationships you will practice, so you know where this worksheet is heading:

Experiment	Case Study	Relationship	New Concept
1	Laptop – Processor	Aggregation 1-1	constructor & setter injection
2	Customer – Car – Driver	Multiple aggregation	1 class related to >1 other class
3	Train – Employee	Aggregation, 2 roles	role name, null, NullPointerException
4	Carriage – Seat – Passenger	Composition + Aggregation	array of objects, multiplicity, guard clause
5	Car – Engine	Composition	the object is created inside the constructor, with no setter
6	Laptop – Printer	Dependency (uses-a)	the object is only passed as a parameter, never stored

C. Work Steps

Each Experiment below stands on its own (separate package) and is broken into small steps. Do them in order; every step builds on the step before it within the same Experiment.

Experiment 1: One-to-One Aggregation (Laptop and Processor)

A Laptop has a Processor. Looking closer, a Processor has its own attributes (brand, cache), which means a Processor is an object of its own class, not just a String or a number. Because the attribute proc in Laptop has the type Processor, we say that the Laptop class has a (has-a) relationship with the Processor class.

Step 1: Class diagram

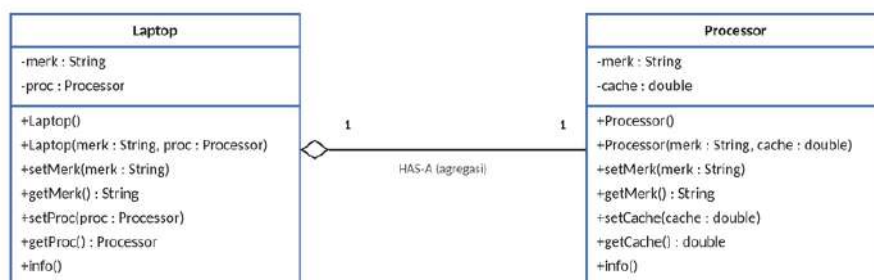


Figure 1: Class diagram of Laptop and Processor. The diamond $\diamond$ on the Laptop side shows that Laptop is the “whole” and Processor is the “part”. Multiplicity 1-1: one Laptop has exactly one Processor.

Concept Note: An empty diamond $\diamond$ = Aggregation: the part (Processor) can still exist on its own even if the whole (Laptop) is deleted. We will prove this in code: watch which line creates the Processor object. Inside class Laptop, or outside it?

Step 2: Create the Processor class

Create the package `id.ac.polinema.classrelation.experiment1`, then create the class `Processor` with the two private attributes below:

```
1 package id.ac.polinema.classrelation.experiment1;
2
3 public class Processor {
4     private String brand;
5     private double cache;
6 }
```

Code 1: Processor.java — the brand and cache attributes

Step 3: Add the Processor constructors

Add a default constructor and a parameterized constructor:

```
1 package id.ac.polinema.classrelation.experiment1;
2
3 public class Processor {
4     private String brand;
5     private double cache;
6
7     public Processor() {
8     }
9
10    public Processor(String brand, double cache) {
11        this.brand = brand;
12        this.cache = cache;
13    }
14 }
```

Code 2: Processor.java with two constructors

Step 4: Add setters, getters, and info()

```
1 package id.ac.polinema.classrelation.experiment1;
2
3 public class Processor {
4     private String brand;
5     private double cache;
6
7     public Processor() {
8     }
9
10    public Processor(String brand, double cache) {
11        this.brand = brand;
12        this.cache = cache;
13    }
14
15    public void setBrand(String brand) {
16        this.brand = brand;
17    }
18
19    public String getBrand() {
20        return brand;
21    }
22
23    public void setCache(double cache) {
24        this.cache = cache;
25    }
26
27    public double getCache() {
28        return cache;
29    }
30
31    public void info() {
32        System.out.printf("Processor Brand = %s\n", brand);
33        System.out.printf("Cache Memory = %.2f\n", cache);
34    }
35 }
```

Code 3: The complete Processor.java with setters, getters, and info()

⚠ If it fails: if VSCode underlines %s/%.2f in red, check your quotation marks and semicolons first — printf is very sensitive to an incomplete format string.

Step 5: Create the Laptop class

Now create the Laptop class in the same package. Look carefully at its two attributes:

```
1 package id.ac.polinema.classrelation.experiment1;
2
3 public class Laptop {
4     private String brand;
5     private Processor proc;
6 }
```

Code 4: Laptop.java — the attributes brand (String) and proc (Processor)

📌 **Concept Note:** brand is a String, but proc is a Processor — an object, not a primitive type. The line `private Processor proc;` is what states in code that “Laptop has-a Processor”.

Step 6: Add the Laptop constructor, setters, and getters

```
1 package id.ac.polinema.classrelation.experiment1;
2
3 public class Laptop {
4     private String brand;
5     private Processor proc;
6
7     public Laptop() {
8     }
9
10    public Laptop(String brand, Processor proc) {
11        this.brand = brand;
12        this.proc = proc;
13    }
14
15    public void setBrand(String brand) {
16        this.brand = brand;
17    }
18
19    public String getBrand() {
20        return brand;
21    }
22
23    public void setProc(Processor proc) {
24        this.proc = proc;
25    }
26
27    public Processor getProc() {
28        return proc;
29    }
30 }
```

Code 5: Laptop.java with constructor, setters, and getters

Step 7: Add the info() method with delegation

```
1     public void info() {
2         System.out.println("Laptop Brand = " + brand);
3         proc.info();
4     }
```

Code 6: The extra info() method in Laptop.java (inserted before the closing brace of the class)

 **Concept Note:** `proc.info()` is delegation: Laptop does not print the Processor details itself; instead it “asks” the Processor object it owns to print itself. This is a common pattern in has-a relationships: the whole calls a method that belongs to its part.

Step 8: Test with the parameterized constructor

Create the class `MainExperiment1` containing a `main()` method:

```
1 package id.ac.polinema.classrelation.experiment1;
2
3 public class MainExperiment1 {
4     public static void main(String[] args) {
5         Processor p = new Processor("Intel i5", 3);
6         Laptop l = new Laptop("Thinkpad", p);
7         l.info();
8     }
9 }
```

Code 7: MainExperiment1.java — creating a Processor and injecting it into a Laptop

 **Checkpoint: the output shows:**

Laptop Brand = Thinkpad, then Processor Brand = Intel i5 and Cache Memory = 3.00.

Step 9: Test with the default constructor and setter injection

Add the following lines at the end of `main()` (do not delete the earlier lines):

```
1 Processor p1 = new Processor();
2 p1.setBrand("Intel i5");
3 p1.setCache(4);
4 Laptop l1 = new Laptop();
5 l1.setBrand("Thinkpad");
6 l1.setProc(p1);
7 l1.info();
```

Code 8: Addition to MainExperiment1.java — creating the object with the default constructor, then filling it through setters

 **Checkpoint: a second output line appears, exactly the same as the first except that cache is 4.00.**

Step 10: Creating the Processor object directly as an argument (anonymous object)

Add one more line at the end of `main()`:

```
1 Laptop l2 = new Laptop("Thinkpad",
2     new Processor("Intel i5", 3));
3 l2.info();
```

Code 9: Addition to MainExperiment1.java — the Processor object created inline as an argument

 **Concept Note:** The output is identical to Step 8, even though `new Processor(...)` is written directly inside the argument. What makes this relationship Aggregation is not where the object is written (in a separate variable or inline), but which class calls `new Processor(...)`. Here the caller is code in `MainExperiment1`, not code inside the `Laptop` class itself. Compare this with Experiment 5 later on.

Questions for Experiment 1

1. Inside the Processor class and the Laptop class there are setter and getter methods for each attribute. What are those setter and getter methods for?
2. Inside the Processor class and the Laptop class there is both a default constructor and a parameterized constructor. How does the use of these two kinds of constructor differ?
3. Look at the Laptop class: of its two attributes (brand and proc), which one has an object type? Which line of code shows that the Laptop class has a relationship with the Processor class?
4. Look at the Laptop class: what is the purpose of the syntax `proc.info()`?
5. In Step 8, the object `p` is created first and then passed to the Laptop constructor. In Step 10, the Processor object is created directly inside the Laptop constructor argument (without a variable `p`). Do the two produce different output? Why?
6. In terms of code, is the Laptop-Processor relationship in this experiment an Aggregation or a Composition? Point to the line of code that proves your answer.
7. Suppose the Laptop constructor were changed as follows, so that the Processor is created inside Laptop instead of being received as a parameter:

```
public Laptop (String brand) {  
    this.brand = brand;  
    this.proc = new Processor ("Generic", 1);  
}
```

Is the Laptop-Processor relationship in this version still an Aggregation? Explain your reasoning (we will prove this answer in code in Experiment 5).

Experiment 2: Aggregation with Multiple Relationships (Car Rental)

A car rental system: a Customer (customer) can rent a Car (car) together with a Driver (driver). The driver fee and the car rental fee are both charged per day. Unlike Experiment 1, here one class (Customer) is related to two other classes at the same time.

Step 1: Class diagram

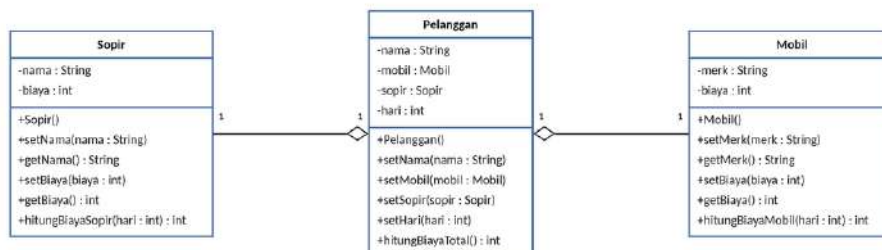


Figure 2: Class diagram of Driver, Customer, and Car. The diamond $\diamond$ is on the Customer side in both relationships. Customer is the “whole” for both Car and Driver.

⚠ If it fails: the correct attribute in Car is brand, not “name”. Make sure the diagram and the code consistently use brand/setBrand/getBrand, matching the sample code below.

Step 2: Create the Car class

Create the package `id.ac.polinema.classrelation.experiment2`, then create the class `Car` with a default constructor, setters/getters, and `calculateCarCost`:

```

1 package id.ac.polinema.classrelation.experiment2;
2
3 public class Car {
4     private String brand;
5     private int cost;
6
7     public Car() {
8     }
9
10    public void setBrand(String brand) {
11        this.brand = brand;
12    }
13
14    public String getBrand() {
15        return brand;
16    }
17
18    public void setCost(int cost) {
19        this.cost = cost;
20    }
21
22    public int getCost() {
23        return cost;
24    }
25
26    public int calculateCarCost(int days) {
27        return cost * days;
28    }
29 }

```

Code 10: The complete Car.java

Step 3: Create the Driver class

```

1 package id.ac.polinema.classrelation.experiment2;
2
3 public class Driver {
4     private String name;
5     private int cost;
6
7     public Driver() {
8     }
9
10    public void setName(String name) {
11        this.name = name;
12    }
13
14    public String getName() {
15        return name;
16    }
17
18    public void setCost(int cost) {
19        this.cost = cost;
20    }
21
22    public int getCost() {
23        return cost;
24    }
25
26    public int calculateDriverCost(int days) {
27        return cost * days;
28    }
29 }

```

Code 11: The complete Driver.java (its structure is deliberately similar to Car.java)

 **Info:** Driver and Car have a similar shape (the fee is charged per day), but they are different classes. Customer will store a reference to each of them separately.

Step 4: Create the Customer class (the whole for Car and Driver)

```
1 package id.ac.polinema.classrelation.experiment2;
2
3 public class Customer {
4     private String name;
5     private Car car;
6     private Driver driver;
7     private int days;
8
9     public Customer() {
10    }
11
12    public void setName(String name) {
13        this.name = name;
14    }
15
16    public String getName() {
17        return name;
18    }
19
20    public void setCar(Car car) {
21        this.car = car;
22    }
23
24    public Car getCar() {
25        return car;
26    }
27
28    public void setDriver(Driver driver) {
29        this.driver = driver;
30    }
31
32    public Driver getDriver() {
33        return driver;
34    }
35
36    public void setDays(int days) {
37        this.days = days;
38    }
39
40    public int getDays() {
41        return days;
42    }
43 }
```

Code 12: Customer.java — two attributes with object types (car, driver)

■ **Concept Note:** Customer has two object-typed attributes at once: car and driver. This means Customer has an Aggregation with two different classes at the same time. That is what makes Experiment 2 different from Experiment 1.

Step 5: Add calculateTotalCost()

```
1 public int calculateTotalCost() {
2     return car.calculateCarCost(days) + driver.calculateDriverCost(days);
3 }
```

Code 13: Addition to Customer.java (insert it before the closing brace of the class)

■ **Concept Note:** car.calculateCarCost(days) calls a method belonging to the car object that Customer holds, and the result is then added to the result of driver.calculateDriverCost(days). Customer itself does not know how to calculate the car or driver fee. It simply asks the right object to calculate it.

Step 6: Test it in MainExperiment2

```
1 package id.ac.polinema.classrelation.experiment2;
2
3 public class MainExperiment2 {
4     public static void main(String[] args) {
5         Car m = new Car();
6         m.setBrand("Avanza");
7         m.setCost(350000);
8
9         Driver s = new Driver();
10        s.setName("John Doe");
11        s.setCost(200000);
12
13        Customer p = new Customer();
14        p.setName("Jane Doe");
15        p.setCar(m);
16        p.setDriver(s);
17        p.setDays(2);
18
19        System.out.println("Total Cost = " + p.calculateTotalCost());
20    }
21 }
```

Code 14: MainExperiment2.java

✓ Checkpoint: the output shows Total Cost = 1100000 (= (350000 + 200000) × 2).

⚠ If it fails: if a `NullPointerException` appears on the `calculateTotalCost()` line, check whether `p.setCar(m)` and `p.setDriver(s)` were really called before `p.calculateTotalCost()`. Without them, car/driver inside `Customer` are still null. We will look at this error more closely in Experiment 3.

Step 7: Navigating a chain of objects (a.b().c())

Add one more line at the end of `main()`:

```
1 System.out.println(p.getCar().getBrand());
```

Code 15: Addition to MainExperiment2.java

✓ Checkpoint: the new line shows Avanza.

Questions for Experiment 2

1. Look at the `Customer` class. Which line of the program shows that the `Customer` class has a relationship with the `Car` class and the `Driver` class?
2. Look at the method `calculateDriverCost` in `Driver` and `calculateCarCost` in `Car`. Why must these methods take `days` as an argument, when `days` is an attribute of `Customer` and not of `Car` or `Driver`?
3. Look at the code of the `Customer` class. What are the statements `car.calculateCarCost(days)` and `driver.calculateDriverCost(days)` for?
4. Look at the `MainExperiment2` class. What are `p.setCar(m)` and `p.setDriver(s)` for?
5. What does `p.calculateTotalCost()` do?
6. In Step 7, `p.getCar().getBrand()` calls two methods in a chain. Explain the order of execution: which object does `p.getCar()` return, and which object then has `.getBrand()` called on it?

7. Suppose `p.setCar(m)` is never called and then `p.calculateTotalCost()` is run — what error will appear? Explain why that error happens, linking it to the idea of object references we studied earlier.

Experiment 3: Aggregation with Two Roles to the Same Class (Train)

A Train is operated by a train driver and an assistant driver. Both of them are employees (Employee) of PT. Kereta Api Indonesia, which means the train driver and the assistant are both of type Employee — only their role differs.

Info: This is not an example of one-to-many multiplicity (that comes in Experiment 4). Here there are two separate relationships to the same Employee class, each with multiplicity 1 — not one relationship with multiplicity 2. The difference matters: if tomorrow a train needs 3 conductors, that would require an array (like Experiment 4), not extra attributes named `assistant2`, `assistant3`, and so on.

Step 1: Class diagram

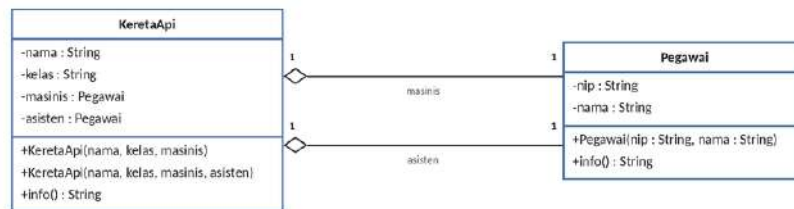


Figure 3: Two separate relationship lines from Train to Employee — one acting as “trainDriver”, the other as “assistant”. Both have multiplicity 1-1.

Step 2: Create the Employee class

```
1 package id.ac.polinema.classrelation.experiment3;
2
3 public class Employee {
4     private String employeeId;
5     private String name;
6
7     public Employee(String employeeId, String name) {
8         this.employeeId = employeeId;
9         this.name = name;
10    }
11
12    public void setEmployeeId(String employeeId) {
13        this.employeeId = employeeId;
14    }
15
16    public String getEmployeeId() {
17        return employeeId;
18    }
19
20    public void setName(String name) {
21        this.name = name;
22    }
23
24    public String getName() {
25        return name;
26    }
27
28    public String info() {
29        String info = "";
30        info += "Employee ID: " + this.employeeId + "\n";
31        info += "Name: " + this.name + "\n";
```

```

32     return info;
33 }
34 }

```

Code 16: The complete *Employee.java*. Note: this class has no default constructor — *employeeId* and *name* must be supplied when the object is created.

⚠ If it fails: because *Employee* only has a parameterized constructor, *new Employee()* (with no arguments) will not compile. This is intentional, to make sure every *Employee* always has an *employeeId* and a *name* from the moment it is created.

Step 3: Create the *Train* class

```

1 package id.ac.polinema.classrelation.experiment3;
2
3 public class Train {
4     private String name;
5     private String trainClass;
6     private Employee trainDriver;
7     private Employee assistant;
8 }

```

Code 17: *Train.java* — two attributes of type *Employee*, distinguished by their names (*trainDriver*, *assistant*)

Step 4: Add two *Train* constructors

One constructor without an assistant (3 parameters) and one with an assistant (4 parameters). This experiment will show what happens if assistant is never filled in:

```

1 public Train(String name, String trainClass, Employee trainDriver) {
2     this.name = name;
3     this.trainClass = trainClass;
4     this.trainDriver = trainDriver;
5 }
6
7 public Train(String name, String trainClass, Employee trainDriver, Employee assistant) {
8     this.name = name;
9     this.trainClass = trainClass;
10    this.trainDriver = trainDriver;
11    this.assistant = assistant;
12 }

```

Code 18: Addition to *Train.java* — the 3-parameter and 4-parameter constructors

Step 5: Add setters/getters and *info()*

```

1 public void setTrainDriver(Employee trainDriver) {
2     this.trainDriver = trainDriver;
3 }
4
5 public Employee getTrainDriver() {
6     return trainDriver;
7 }
8
9 public void setAssistant(Employee assistant) {
10    this.assistant = assistant;
11 }
12
13 public Employee getAssistant() {
14    return assistant;
15 }
16
17 public String info() {
18    String info = "";
19    info += "Name: " + this.name + "\n";

```

```

20     info += "Class: " + this.trainClass + "\n";
21     info += "Train Driver: " + this.trainDriver.info() + "\n";
22     info += "Assistant: " + this.assistant.info() + "\n";
23     return info;
24 }

```

Code 19: Addition to Train.java — setters, getters, and info()

Step 6: Test it with both attributes filled in

```

1  package id.ac.polinema.classrelation.experiment3;
2
3  public class MainExperiment3 {
4      public static void main(String[] args) {
5          Employee trainDriver = new Employee("1234", "Spongebob Squarepants");
6          Employee assistant = new Employee("4567", "Patrick Star");
7          Train train = new Train("Gaya Baru", "Business", trainDriver, assistant);
8          System.out.println(train.info());
9      }
10 }

```

Code 20: MainExperiment3.java

☑ **Checkpoint: the output shows the full info: Name, Class, Train Driver (Spongebob), and Assistant (Patrick).**

Step 7: Get to know NullPointerException first

Before moving on to the next challenge, understand the pattern: a `NullPointerException` always appears when we call a method on an object reference that is still null (never created with `new`). If `this.assistant` is never filled in, then `this.assistant.info()` is the same as calling a method on “nothing”, and Java will stop the program with this error.

📌 **Concept Note:** The protective pattern is called a **guard clause**: check whether the object reference is null before calling its method, for example if `(this.assistant != null) { ... }`. Remember this pattern — we will use it shortly.

Step 8: Reproduce the bug — a Train with no assistant

Create a new class `MainQuestion` in the same package:

```

1  package id.ac.polinema.classrelation.experiment3;
2
3  public class MainQuestion {
4      public static void main(String[] args) {
5          Employee trainDriver = new Employee("1234", "Spongebob Squarepants");
6          Train train = new Train("Gaya Baru", "Business", trainDriver);
7          System.out.println(train.info());
8      }
9  }

```

Code 21: MainQuestion.java — Train created through the 3-parameter constructor (without an assistant)

⚠ **If it fails: the program will stop with a `NullPointerException` on the line `this.assistant.info()` inside `Train.info()`. This is intentional — move on to the next step to fix it.**

Step 9: Fix it with a guard clause

Change the info() method in Train.java so that the Assistant: ... line is only printed when assistant is actually filled in:

```
1 public String info() {
2     String info = "";
3     info += "Name: " + this.name + "\n";
4     info += "Class: " + this.trainClass + "\n";
5     info += "Train Driver: " + this.trainDriver.info() + "\n";
6     if (this.assistant != null) {
7         info += "Assistant: " + this.assistant.info() + "\n";
8     }
9     return info;
10 }
```

Code 22: Train.java after being fixed with a guard clause

✓ Checkpoint: run MainQuestion again — the program now finishes without an error, and the “Assistant: ...” line is not printed.

Questions for Experiment 3

1. Inside the info() method of the Train class, what are the lines this.trainDriver.info() and this.assistant.info() used for?
2. What is the output of MainQuestion before the fix (Step 8)? Why does that happen?
3. Link this to the topic of object references: what is stored in the assistant variable inside a Train object created with the 3-parameter constructor, before the guard clause is added?
4. After the guard clause is added (Step 9), does the trainDriver object need the same check? Look at both Train constructors — is it possible for trainDriver to be null? Explain.
5. The Employee class is used through two different attributes (trainDriver and assistant) in Train. Does this give Train two different Employee objects, or one Employee object used twice? Explain based on the code in Step 6.

Experiment 4: Array of Objects and Multiplicity (Carriage, Seat, and Passenger)

In Experiments 1-3, every relationship was one-to-one. But sometimes one object really does own many other objects at once, and the number can change — this is called multiplicity. A Carriage (train carriage) has many Seat (seats), and each Seat (once it is taken) has one Passenger (passenger).

This case study is also an important example: two neighbouring has-a relationships (Carriage-Seat and Seat-Passenger) turn out to be of different kinds when we check them against the code criterion, even though at first glance both look like “a collection of objects”. Watch carefully in Steps 3 and 4: who calls new to create the Seat objects, and who calls new to create the Passenger object?

The multiplicity notation in the table below is used in the diagram that follows:

Multiplicity	Meaning
0..1	0 or 1 instance
1	Exactly 1 instance
0..*	0 or more instances
1..*	At least 1 instance

n	Exactly n instances (replace n with a number)
m..n	At least m instances, but no more than n

Step 1: Class diagram

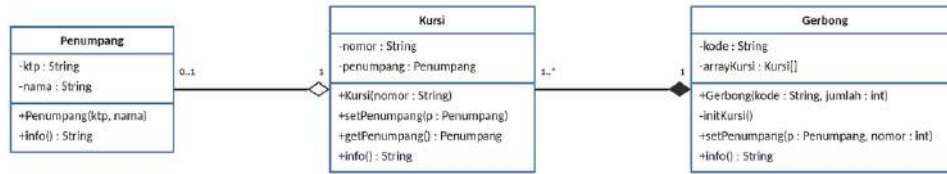


Figure 4: An open diamond $\diamond$ on the Seat side (Seat HAS-A 0..1 Passenger –Aggregation, because a seat may be empty and the Passenger is created outside Seat) versus a filled diamond $\blacklozenge$ on the Carriage side (Carriage HAS-A 1..* Seat –Composition; see Step 4: every Seat is created by Carriage itself through `initSeats()`).

Concept Note: The multiplicity 1..* in Carriage-Seat is different from Experiment 3: there we had 2 separate relationships with fixed names (trainDriver, assistant). Here the number of Seat is not fixed and they do not have individual names. That is why an array is used instead of separately named attributes. The array itself does not decide the kind of relationship (Aggregation or Composition) — that is still decided by the code criterion, as you will see in Step 4.

Step 2: Create the Passenger class

```

1 package id.ac.polinema.classrelation.experiment4;
2
3 public class Passenger {
4     private String idCard;
5     private String name;
6
7     public Passenger(String idCard, String name) {
8         this.idCard = idCard;
9         this.name = name;
10    }
11
12    public String getIdCard() {
13        return idCard;
14    }
15
16    public String getName() {
17        return name;
18    }
19
20    public String info() {
21        String info = "";
22        info += "ID Card: " + idCard + "\n";
23        info += "Name: " + name + "\n";
24        return info;
25    }
26 }
  
```

Code 23: The complete Passenger.java

Step 3: Create the Seat class (has-a 0..1 Passenger)

```

1 package id.ac.polinema.classrelation.experiment4;
2
3 public class Seat {
4     private String number;
5     private Passenger passenger;
6
7     public Seat(String number) {
  
```

```

8      this.number = number;
9  }
10
11  public void setPassenger(Passenger passenger) {
12      this.passenger = passenger;
13  }
14
15  public Passenger getPassenger() {
16      return passenger;
17  }
18
19  public String info() {
20      String info = "";
21      info += "Number: " + number + "\n";
22      if (this.passenger != null) {
23          info += "Passenger: " + passenger.info() + "\n";
24      }
25      return info;
26  }
27  }

```

Code 24: The complete *Seat.java* — note the guard clause *if (this.passenger != null)*, the same pattern we applied ourselves in Experiment 3

Concept Note: Unlike *Train.assistant* (Experiment 3), which may be filled in later but is still considered mandatory, *Seat.passenger* is naturally allowed to be empty. A seat that nobody is sitting in is not a bug — it is a normal condition. So the guard clause here is not a "bug fix" as it was in Experiment 3; it is part of the design from the start.

Step 4: Create the Carriage class (has-a 1..* Seat, Composition)

```

1  package id.ac.polinema.classrelation.experiment4;
2
3  public class Carriage {
4      private String code;
5      private Seat[] seatArray;
6
7      public Carriage(String code, int count) {
8          this.code = code;
9          this.seatArray = new Seat[count];
10         this.initSeats();
11     }
12
13     private void initSeats() {
14         for (int i = 0; i < seatArray.length; i++) {
15             this.seatArray[i] = new Seat(String.valueOf(i + 1));
16         }
17     }
18 }

```

Code 25: *Carriage.java* — the constructor fills the whole *Seat* array directly through *initSeats()*

Concept Note: *initSeats()* is private and is called inside the *Carriage* constructor itself. The caller from outside (*MainExperiment4*) never creates a *Seat* object. All the seats are automatically “ready to use” as soon as the *Carriage* is created. Apply the code criterion from the Learning Outcomes: because *Carriage* itself calls *new Seat(...)* (there is no parameter of type *Seat* or *Seat[]* in its constructor), the *Carriage-Seat* relationship is actually Composition, not Aggregation. We will study the same pattern more formally in Experiment 5; the only difference is that there the part is a single object, while here the part is an array (many *Seat* objects at once).

Step 5: Add a method to navigate to a specific Seat

```
1 public void setPassenger(Passenger passenger, int number) {
2     this.seatArray[number - 1].setPassenger(passenger);
3 }
4
5 public String info() {
6     String info = "";
7     info += "Code: " + code + "\n";
8     for (Seat seat : seatArray) {
9         info += seat.info();
10    }
11    return info;
12 }
```

Code 26: Addition to Carriage.java

Step 6: Test it in MainExperiment4

```
1 package id.ac.polinema.classrelation.experiment4;
2
3 public class MainExperiment4 {
4     public static void main(String[] args) {
5         Passenger p = new Passenger("12345", "Mr. Krab");
6         Carriage carriage = new Carriage("A", 10);
7         carriage.setPassenger(p, 1);
8         System.out.println(carriage.info());
9     }
10 }
```

Code 27: MainExperiment4.java

☑ Checkpoint: the output prints 10 Seat lines (Number: 1 to 10); only Seat number 1 shows a "Passenger: ..." line (Mr. Krab).

⚠ If it fails: an `ArrayIndexOutOfBoundsException` means we are filling an index that does not exist, for example `carriage.setPassenger(p, 11)` when the array only has 10 elements (valid seat numbers are 1 to 10, because `number - 1` is used as the array index, which starts at 0).

Questions for Experiment 4

1. In the main program inside the `MainExperiment4` class, how many seats are there in Carriage A?
2. Look at the snippet `if (this.passenger != null) { ... }` in the `info()` method of the `Seat` class. What does that code mean?
3. In the `setPassenger()` method of the `Carriage` class, why is 1 subtracted from number?
4. Create a new object `budi` of type `Passenger`, then put it into the carriage with `carriage.setPassenger(budi, 1)`, overwriting Mr. Krab who is already sitting there. What happens? Does Java give a warning or an error?
5. Modify the program so that a seat that already has a passenger cannot be taken by someone else (add a check in `Carriage.setPassenger()` before the line `seatArray[number - 1].setPassenger(...)` runs).
6. Compare the three has-a relationships we have practiced: Laptop-Processor (Experiment 1, 1-1), Train-Employee (Experiment 3, two named 1-1 relationships), and Carriage-Seat (Experiment 4, 1..*). In what kind of case would we choose an array, and in what kind of case would we choose individually named attributes?
7. Apply the code criterion (who calls new) to the two has-a relationships in this experiment: Carriage-Seat and Seat-Passenger. Which one is Aggregation and which one is Composition? Point to the line of code that proves each answer.

Experiment 5: Composition (Car and Engine)

Think back to Experiment 4: the `initSeats()` method in `Carriage` calls `new Seat()` itself and never receives a `Seat` object from outside. By the code criterion, that is already Composition — it simply was not named explicitly there. Experiments 1-3 were pure Aggregation: the “part” object was always created outside the whole class and then injected through a constructor or a setter. Now it is your turn to practice the Composition pattern deliberately, with a new case study: a `Car` that creates its own `Engine` and never receives one from outside.

Step 1: Class diagram

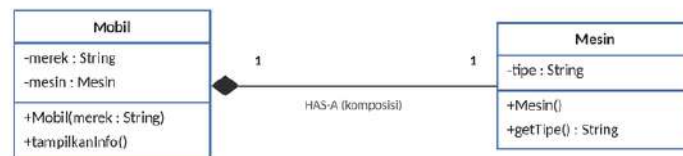


Figure 5: A filled diamond $\blacklozenge$ = Composition. Note: the `Car` class has no `setEngine()` — this is on purpose, not an oversight.

Concept Note: A filled diamond $\blacklozenge$ means exclusive ownership and a tied lifecycle: if the `Car` object is deleted or lost, its `Engine` object goes with it, because nobody other than that `Car` ever held a reference to the `Engine`.

Step 2: Create the Engine class

```
1 package id.ac.polinema.classrelation.experiment5;
2
3 public class Engine {
4     private String type;
5
6     public Engine() {
7         this.type = "4-cylinder";
8     }
9
10    public String getType() {
11        return type;
12    }
13 }
```

Code 28: `Engine.java` — the default constructor sets the value of `type` directly

Info: `Engine` is deliberately kept as simple as possible (no parameterized constructor, no setter) so that our attention stays on the `Car` class in the next step.

Step 3: Create the Car class — Engine is created inside the constructor

```
1 package id.ac.polinema.classrelation.experiment5;
2
3 public class Car {
4     private String brand;
5     private Engine engine;
6
7     public Car(String brand) {
8         this.brand = brand;
9         this.engine = new Engine();
10    }
11 }
```

Code 29: `Car.java` — the `Car` constructor itself calls `new Engine()`

■ **Concept Note:** Compare this with `Laptop(String brand, Processor proc)` in Experiment 1: there the Processor was received as a parameter. Here the Car constructor only takes the brand, and there is no parameter of type Engine at all. The Engine is created by Car itself, using `new Engine()` on a line of code that belongs to the Car class. This is the code criterion that separates Aggregation from Composition.

Step 4: Add `displayInfo()`, deliberately without an Engine getter/setter

```
1 public void displayInfo() {
2     System.out.println("Car: " + brand);
3     System.out.println("Engine: " + engine.getType());
4 }
```

Code 30: Addition to Car.java (insert it before the closing brace of the class)

⚠ If it fails: do not add `setEngine(Engine engine)` or `getEngine(): Engine` to this class, even if VSCode suggests it through auto-generated getters/setters. Without a setter, there is no way for code outside Car to replace the Engine that was created. That is what keeps this relationship a Composition (see Question no. 2 below).

Step 5: Test it in MainExperiment5

```
1 package id.ac.polinema.classrelation.experiment5;
2
3 public class MainExperiment5 {
4     public static void main(String[] args) {
5         Car car = new Car("Avanza");
6         car.displayInfo();
7     }
8 }
```

Code 31: MainExperiment5.java

✓ **Checkpoint:** the output shows `Car: Avanza`, then `Engine: 4-cylinder` — even though we never wrote `new Engine()` anywhere in `MainExperiment5`.

Questions for Experiment 5

1. In the Car class, which line shows that the Engine is a part “owned exclusively” by the Car (and not merely “borrowed”)?
2. From a design point of view, what happens if a `setEngine(Engine engine)` method is added to the Car class? Would the relationship still be a Composition? Explain.
3. Compare this with Experiment 1 (Laptop-Processor): name one difference in the code that makes one of them an Aggregation and the other a Composition.
4. If the car object in MainExperiment5 is set to null after `displayInfo()` is called, what happens to its Engine object? Compare this with what would happen to the Processor object in Experiment 1 if its Laptop object were deleted — could that Processor still be “rescued” by other code? Why can the Engine not be?
5. Try this separately (in your own file or package): add a second constructor to Car that takes an Engine parameter, similar to the pattern in Experiment 1: `public Car(String brand, Engine engine) {`

this.brand = brand; this.engine = engine; }. If this constructor is used, does Car-Engine become an Aggregation? Explain your reasoning.

Experiment 6: Dependency / Uses-A (A Laptop Printing a Document on a Printer)

The last relationship we need to tell apart from Aggregation and Composition is Dependency (uses-a): one class uses another class only briefly, through a method parameter, without ever storing it as an attribute.

Step 1: Class diagram



Figure 6: A dashed line with an open arrowhead (not a diamond!) indicates a Dependency: the Laptop “briefly uses” a Printer object without storing it.

Concept Note: Notice that there is no Printer attribute in the Laptop class. This is the basic difference from Aggregation and Composition, which both store the part object as an attribute. A Dependency only “borrows” the object for the duration of a single method call.

Step 2: Create the Printer class

```
1 package id.ac.polinema.classrelation.experiment6;
2
3 public class Printer {
4     private String brand;
5
6     public Printer(String brand) {
7         this.brand = brand;
8     }
9
10    public void print(String fileName) {
11        System.out.println("[ " + brand + " ] Printing " + fileName + "...");
12        System.out.println("[ " + brand + " ] Done.");
13    }
14 }
```

Code 32: The complete Printer.java

Step 3: Create the Laptop class with a printDocument() method

This time the Laptop is kept simple (only the brand attribute), plus one method that takes a Printer as a parameter:

```
1 package id.ac.polinema.classrelation.experiment6;
2
3 public class Laptop {
4     private String brand;
5
6     public Laptop(String brand) {
7         this.brand = brand;
8     }
9
10    public void printDocument(Printer printer, String fileName) {
11        System.out.println(brand + " is sending a document to the printer...");
12        printer.print(fileName);
13    }
14 }
```

Code 33: Laptop.java — Printer appears only as a method parameter and never becomes an attribute

■ **Concept Note:** Once `printDocument()` has finished running, the Laptop no longer “knows” the printer object it just used. A method parameter only lives while that method is running. If we want the Laptop to remember the same printer so it can be used again and again, we have to store it as an attribute (see Question no. 4). And as soon as that happens, the relationship is no longer a Dependency.

Step 4: Test it in MainExperiment6

```
1 package id.ac.polinema.classrelation.experiment6;
2
3 public class MainExperiment6 {
4     public static void main(String[] args) {
5         Laptop laptop = new Laptop("Thinkpad");
6         Printer printer = new Printer("Epson L3110");
7         laptop.printDocument(printer, "Report.pdf");
8     }
9 }
```

Code 34: MainExperiment6.java

☑ **Checkpoint:** the output shows three lines: “Thinkpad is sending a document to the printer...”, “[Epson L3110] Printing Report.pdf...”, and “[Epson L3110] Done.”.

Questions for Experiment 6

1. Does the Laptop class in this experiment have an attribute of type Printer? Compare it with Experiment 1, where the Processor is stored as an attribute of Laptop.
2. After the `printDocument()` method has finished running, does the Laptop still hold a reference to the printer object it used? Explain based on the lines of code in the Laptop class.
3. Why is the Laptop-Printer relationship in this experiment called a Dependency (uses-a) rather than an Aggregation, even though both involve two objects interacting?
4. Try changing the Laptop code so that the Printer is stored as an attribute (e.g. `private Printer defaultPrinter`, filled through a constructor or a setter, then reused in `printDocument()` without a Printer parameter). Does the relationship now change from Dependency to Aggregation? Explain.
5. Complete the following table in your own words (you may answer it in your report): for each of Aggregation, Composition, and Dependency, state (a) whether the part object is stored as an attribute or not, and (b) who calls new to create that part object.

D. Assignment and Deliverables

Submit the following in the format requested by your lecturer:

- A screenshot of the program output for every Experiment (1 to 6).
- The source code of all packages, `classrelation.experiment1` through `experiment6`.

Independent assignment:

1. Design a case study of your own (any topic — for example a library, a clinic, an online shop, and so on), draw its class diagram, and then implement it as a program. The case study must involve at least 4 classes (the class containing main does not count) and must include all three: at least one Aggregation, one Composition, and one Dependency. In your report, mark which part of the code is which kind of relationship, and give your reasons.

2. Answer briefly (3-5 sentences): when designing your own new system, how do we decide whether a relationship between classes should be an Aggregation, a Composition, or a Dependency? State the key questions we ask ourselves when deciding.

 **Info:** The independent assignment deliberately requires all three kinds of relationship (not only Aggregation, as in the earlier version of this worksheet), so that we really practice the full range of has-a relationships discussed in class, not just one kind.

E. Assessment Criteria

Component	Weight	Full Criteria (100%)	Minimum Criteria
Work steps for Experiments 1-6 completed	35%	All steps carried out and working as described in the checkpoints	Most steps completed; the final result of each Experiment runs
Checkpoints verified	20%	All checkpoints reached and proven (screenshot/output)	Some checkpoints proven
Comprehension questions answered	20%	All questions (including the ones marked "new") answered with sound reasoning and reference to the relevant lines of code	Most questions answered; some reasoning is not quite right
Independent assignment	25%	Case study is correct, at least 4 classes, all three kinds of relationship (Aggregation, Composition, Dependency) present and correctly marked	A case study exists but does not yet cover all three kinds of relationship, or the explanation is incomplete